

시각화 후보

확인해보니 시각화할 만한 게 꽤 많아. 특히 `artifacts` 에는 바로 그래프로 만들 수 있는 `training_summary.json`, `checkpoint_evaluations.json`, `monitor_*.csv`, `analysis/*_episodes.csv`, `analysis/*_contacts.csv`, `analysis/*_summary.json`, TensorBoard event 파일이 있어.

1순위로 넣기 좋은 것

1. 학습 진행 곡선

- 내용: timestep별 `mean_return`, `mean_useful_bounces`, `30+ bounce rate`
- 위치: 제어 모델과 학습
- 데이터: `_checkpoint_evaluations.json`
- 예: v25는 checkpoint 0, 100k, 200k... 500k 평가가 있음.

2. 모델 버전별 성능 비교

- 내용: v25, v26, v30, v31, v32의 `mean useful bounces`, `max useful bounces`, `30+ rate`
- 위치: 제어 모델과 학습
- 데이터: `analysis/*_summary.json`
- 특히 v25뿐 아니라 v26/v30/v32 쪽 비교가 보기 좋음.

3. 실패 원인 breakdown

- 내용: `time_limit`, `low_apex_contact`, `ball_out_of_bounds`, `floor_contact` 비율
- 위치: 보상 설계 또는 제어 모델과 학습
- 데이터: `failure_counts` in `_summary.json`
- 왜 좋은가: 모델이 "어디서 무너지는지" 바로 보임.

4. episode별 성능 분포

- 내용: 100 episode의 useful bounces histogram, return 분포, steps 분포
- 위치: 제어 모델과 학습
- 데이터: `analysis/*_episodes.csv`
- 예: `episode`, `return`, `steps`, `contact_count`, `useful_bounces`, `stable_cycles`, `failure_reason`

5. return vs useful bounces scatter

- 내용: 각 episode가 높은 return을 받은 이유가 bounce 수와 얼마나 연결되는지
- 위치: 제어 모델과 학습
- 데이터: `_episodes.csv`

문서 이해를 돕는 구조 시각화

6. 전체 실행 구조 다이어그램

- 내용: 서버 시뮬레이션 → WebSocket 상태 → MuJoCo WASM/Three.js 렌더링
- 위치: 개요
- 데이터 필요 없음. 현재 docs 내용을 도식화.

1. Observation 55D 구성 stacked bar

- 내용: joint, racket, ball, phase, contact context, next intercept 등 입력 구성
- 위치: 상태와 행동
- 데이터: docs 표 기반.

2. Action 15D 구성 stacked bar

- 내용: position residual, tilt residual, velocity residual, apex timing residual
- 위치: 상태와 행동
- 데이터: docs 표 기반.

3. Reward term map

- 내용: contact bonus, apex match, easy next ball, stable cycle, penalties 가 어떤 목적을 갖는지
- 위치: 보상 설계
- 데이터: docs + env config.

contact CSV로 만들 수 있는 고급 시각화

10. contact별 apex 품질 그래프

- 내용: contact index에 따른 projected apex height, target height, shortfall
- 위치: 보상 설계
- 데이터: `_contacts.csv`

1. desired vs actual outgoing velocity

- 내용: 접촉 후 원하는 공 속도와 실제 공 속도 비교

- 위치: 보상 설계 또는 상태와 행동

- 데이터: `desired_outgoing_velocity_*`, `actual_outgoing_velocity_*`

1. next intercept quality

- 내용: 다음 공이 얼마나 치기 쉬운 상태인지 `easy_next_ball_score`, `next_intercept_xy_error`

- 위치: 보상 설계

- 데이터: `_contacts.csv`

1. action residual 분포

- 내용: 15D action이 어느 축에서 많이 쓰였는지

- 위치: 상태와 행동

- 데이터: `applied_action_0_radial` ... `applied_action_14_strike_plane_z_residual`

1. stable cycle 누적 그래프

- 내용: contact가 반복될수록 stable cycle이 얼마나 유지되는지

- 위치: 제어 모델과 학습

- 데이터: `stable_cycle_count`, `consecutive_stable_cycle_count`

나중에 해도 좋은 것

15. TensorBoard PPO 내부 곡선

- 내용: policy loss, value loss, entropy, approx KL, explained variance
- 위치: 별도 "Training Diagnostics" 섹션
- 데이터: `tb/PP0_1/events.out.tfevents...`
- 단점: 변환 스크립트가 필요함.

1. 환경 파라미터 카드/도식

- 내용: timestep, gravity, ball mass/radius, racket size, target height
- 위치: 시뮬레이션 환경
- 데이터: `scene.xml`, training summary.

내 추천은 우선 1, 2, 3, 4, 7, 8 부터 넣는 거야. 이 조합이면 "어떤 RL 구조로 했고, 학습이 실제로 좋아졌고, 지금 실패는 어디서 나는지"가 Docs에서 바로 보인다.